ICT103 Programming

Topic 5
Data Structures: Lists, Dictionaries and Introduction to DataFrames (Pandas)

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 4:

- Understand the need for the repetition control structure
- Explore various types of repetition control structures
  - for loop, nested loop, pre-test loops
- Advantages and disadvantages of the various structures

Week 5: Lesson Learning Outcomes

- Create and use Lists, Dictionaries, and DataFrames
- Perform traversal on lists, dictionaries, and dataframes
- Apply common functions/operations
- Understand real-life analogies for each structure

What is a Data Structure?

- A way to store and organize data for easy access and modification.
- Examples: Lists, Dictionaries, Tuples, DataFrames
- Real-life analogy: A shelf (**structure**) to store books (**data**)

List Basics

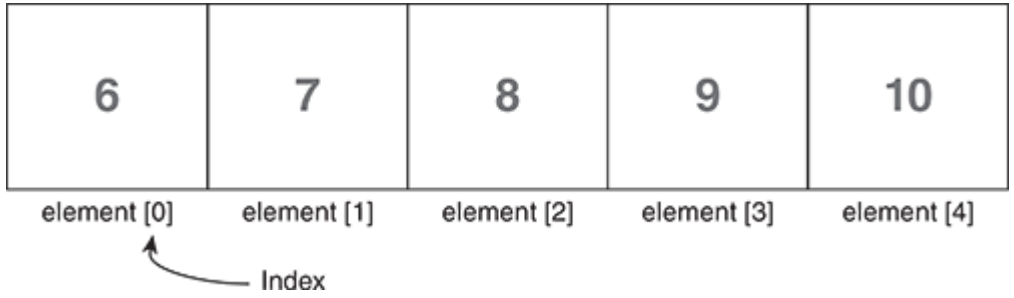
- List
  - A data structure that stores a collection of elements in a specific order or location
- Classified as composite data types because they are constructed from primitive data types, such as integers

- Three types:
  - A one-dimensional list is linear.
  - A two-dimensional list has rows and columns that form a grid, or matrix.
  - A multi-dimensional list is a list of lists

List Characteristics

- Heterogeneous
  - can contain any combination of data types such as strings, integers, floats, and Booleans

- Ordered
  - Store consecutive data where each element is identified by a sequential list index (subscript) enclosed in square brackets.



- Infinite
  - Need a size when created (can be even zero), but can grow as large as the computer's memory allows

One-Dimensional List

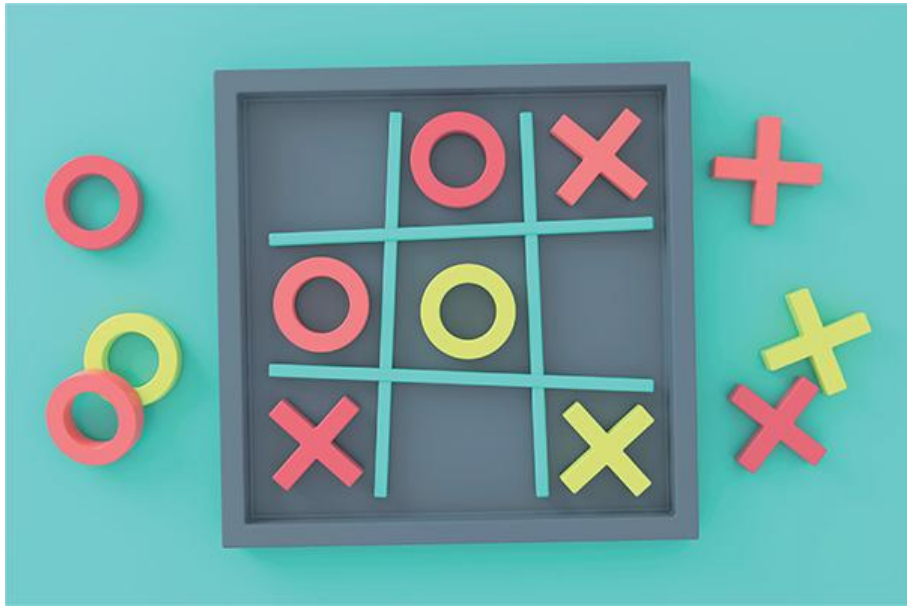
- Initialise Lists
  - Has a name and uses a single index to identify list elements
  - Same naming convention is used when naming a list

Syntax to declare an empty list:

```
empty_list = []
empty_list = list()
```

Syntax to declare and initialize a list:

```
numbers_list = [2, 6, 2, 3, 9, 12]
letters_list = ["A", "H", "C", "B"]
mixed_list = [2, 4, "B", 2, "Z"]
```



" "	"O"	"X"
first_row[0]	first_row[1]	first_row[2]

Output a List Element

- Can output single element of a list using its index value

```
numbers = [6, 7, 8, 9, 10]
print(numbers[3])
OUTPUT:
9
```

- Indexes can also be negative numbers
  - Last element in a list is at index -1, second to last is at -2 and so on.

```
numbers = [6, 7, 8, 9, 10]
print(numbers[-3])
OUTPUT:
8
```

Index Errors

- Index Errors
  - Using an index value that is too large for the list produces an index error or a runtime warning
  - Index errors are sometimes referred to as boundary errors, out-of-bounds exceptions, or subscript errors

Python 3.7.2 Shell

File Edit Shell Debug Options Window Help

```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22
:20:52) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" fo
r more information.
>>> numbers = [1, 2, 3, 4, 5]
>>> numbers[6]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    numbers[6]
IndexError: list index out of range
>>>
```

Ln: 9 Col: 4

Traverse a List

- Accessing each list element in sequence
- You can traverse a list to output every element, count the total number of elements, search for an element, or sum the elements
- Two options for traversing a list:
  - Index-based traversal of a list requires a loop counter variable
  - Element-based traversal uses the list as the iterative sequence
    - Instead of using range() use the list by itself

Examples

Index-based traversal

```
fruits = ["apple", "banana", "mango"]
for i in range(len(fruits)):
    print(fruits[i])
```

Output

```
apple
banana
mango
```

Element- based traversal

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

Output

```
apple
banana
mango
```

Input List Elements

- Can create an empty list and then append each new element as it is input
- Use append method to append to a list

```
a_list.append(5)
```

- Two options for telling the program when to stop appending list items:
  - Use a known amount
    - Specifies the number of elements to store ahead of time, either within the program or as another input statement
  - Use sentinel values, or values that signal that the user is done with input

Operations, Functions and Methods

- **Change a list element**
  - Can change the value of **any** list element using an assignment statement

```
numbers = [6, 7, 8, 9, 10]
numbers[3] = 22
print(numbers)
```

OUTPUT:

```
[6, 7, 8, 22, 10]
```

  - Modifying **every** item in a list requires the index-based loop

```
numbers = [6, 7, 8, 9, 10]
for i in range(len(numbers)):
    numbers[i] += 1
print(numbers)
```

OUTPUT:

```
[7, 8, 9, 10, 11]
```

Contd…

- **Find a list element**
  - To determine whether an element is present, you can loop through the list and compare each list item to the target value
  - You can include an accumulator that records the number of times the target element appears in the list

```
count = 0
numbers = [6, 22, 8, 22, 12, 6, 99, 20, 2, 4]
for element in numbers:
    if element == 22:
        count += 1
print("The array contains", count, "instances of 22.")
```

OUTPUT:

```
The array contains 2 instances of 22.
```

- Alternatively, call the **count()** method on the list and pass in the element to count

```
numbers = [6, 22, 8, 22, 12, 6, 99, 20, 2, 4]
total = numbers.count(22)
print("The array contains", total, "instances of 22.")
```

OUTPUT:

```
The array contains 2 instances of 22.
```

Contd…

- **Sum list elements**
  - Use an accumulator to find the sum of the elements in a list

```
total = 0
numbers = [6, 7, 8, 9, 10]
for element in numbers:
    total += element
print("The sum of the array:", total)
```

OUTPUT:

```
The sum of the array: 40
```

  - Use the built-in function **sum()** to find the sum of elements in a list

```
numbers = [6, 7, 8, 9, 10]
total = sum(numbers)
print("The sum of the array:", total)
```

OUTPUT:

```
The sum of the array: 40
```

Common built-in methods and functions

Function or Method	Description	Syntax	Output
min()	Find the minimum value in a list.	min([2,5,3,1])	1
max()	Find the maximum value in a list.	max([2,5,3,1])	5
sum()	Add all elements in the list together.	sum([2,5,3,1])	11
count()	Count the elements that match the given parameter.	[2,5,3,3,1].count(3)	2
index()	Find the index of the first element which matches the given parameter.	[2,5,3,2,1].index(2)	0
sort()	Sort items	numbers = [4,2,3,5,1] numbers.sort()	[1,2,3,4,5]
insert(i,x)	Add item at index	numbers.insert(1,6)	[1,6,2,3,4,5]
remove(x)	Remove item	numbers.remove(4)	[1,6,2,3,5]
pop(i)	Remove item at index	numbers.pop(0)	[6,2,3,5]
len()	Count items	len(numbers)	4
reverse()	Reverse list	numbers.reverse()	[5,3,2,6]

List Comprehension

- A Python technique that places for-loop logic inside the square brackets of a list declaration
- **Dimensional Manipulation**

```
numbers = []
for i in range(100):
    numbers.append(i)
print(numbers)
```

OUTPUT:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46,
47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76,
77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91,
92, 93, 94, 95, 96, 97, 98, 99]
```

Manually creating a list with many numbers

```
numbers = [i for i in range(100)]
print(numbers)
```

OUTPUT:

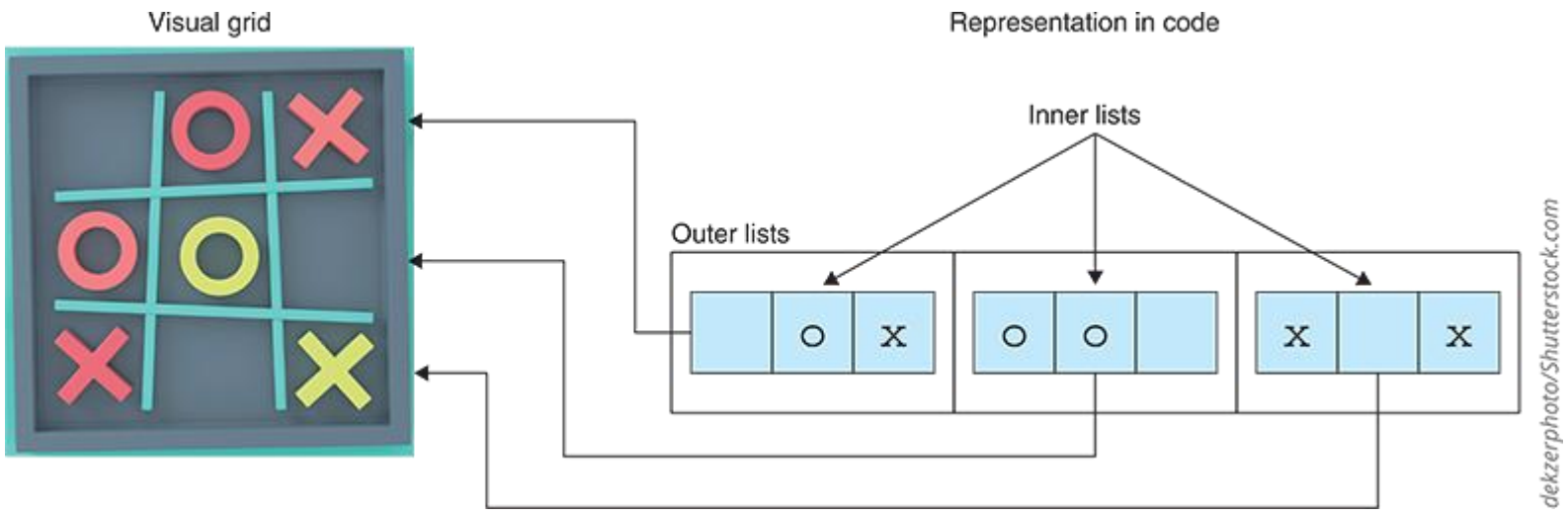
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46,
47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76,
77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91,
92, 93, 94, 95, 96, 97, 98, 99]
```

Using list comprehension to create a list with many numbers

Two-Dimensional List Basics

- Conceptually, two-dimensional arrays appear to be tables, grids, or matrices
- Each cell is referenced by two index values: one to indicate the row and one to indicate the column

	Column 0	Column 1	Column 2
Row 0	" " [0] [0]	"O" [0] [1]	"X" [0] [2]
Row 1	"O" [1] [0]	"O" [1] [1]	" " [1] [2]
Row 2	"X" [2] [0]	" " [2] [1]	"X" [2] [2]



Initialise a Two-Dimensional List

- In Python, you can declare a two-dimensional list using nested square brackets

- Syntax:

```
grid_of_numbers = [[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]]
```

```
matrix = []
rows = int(input("How many rows? "))
columns = int(input("How many columns? "))
for r in range(rows):
    matrix.append([])
    for c in range(columns):
        matrix[r].append(0)
print(matrix)
```

OUTPUT:

```
How many rows? 3
How many columns? 4
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

Traversing a Two-Dimensional List

- Traversing a two-dimensional list typically requires a nested loop
  - Outer loop controls the iteration through each row
  - Inner loop controls the iterations through the columns

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

for row in matrix:
    for value in row:
        print(value, end=" ")
```

Output

```
1 2 3 4 5 6 7 8 9
=== Code Execution Successful ===
```

join() Function

- Used to merge items from a list or tuple into a single string
- Takes separator and combines the elements of the collection with the separator in between.

```python
words = ["Hello", "world", "from", "Python"]

# Join with a space as a separator
result_space = " ".join(words)
print(result_space)  # Output: Hello world from Python

# Join with a comma and space as a separator
result_comma = ", ".join(words)
print(result_comma)  # Output: Hello, world, from, Python

# Join with an empty string (no separator)
result_no_sep = "".join(words)
print(result_no_sep) # Output: HelloworldfromPython
```

Output

```
Hello world from Python
Hello, world, from, Python
HelloworldfromPython
```

Dictionary Basics

- A Python data structure that resembles a list but contains key-value pairs
- Two values must be provided for each dictionary entry:
  - A value to store
  - A key to reference the entry, which is usually a string

```
scores = {"mandy": 2,
          "fernando": 4,
          "jessica": 1,
          "prateek": 1}

print("Mandy's score:", scores["mandy"])
print("Fernando's score:", scores["fernando"])
print("Jessica's score:", scores["jessica"])
print("Prateek's score:", scores["prateek"])
```

OUTPUT:

```
Mandy's score: 2
Fernando's score: 4
Jessica's score: 1
Prateek's score: 1
```

- Entries are enclosed in curly braces (you can use empty curly braces to initialise an empty dictionary)
- Each key-value pair includes a colon between the key and value

Dictionary Access

- You specify the key to access a dictionary value
  - Attempting to access an entry using an absent key causes an error
- You can use keys() method to produce all the keys stored in the dictionary
- You can also access the keys as an iterative sequence using for-loop

```
scores = {"mandy": 2,
          "fernando": 4,
          "jessica": 1,
          "prateek": 1}

keys = scores.keys()

print("Keys:", ", ".join(keys))
```

OUTPUT:

```
Keys: mandy, fernando, jessica, prateek
```

```
scores = {"mandy": 2,
          "fernando": 4,
          "jessica": 1,
          "prateek": 1}

for key in scores.keys():
    print("Name, score:", key + ",", scores[key])
```

OUTPUT:

```
Name, score: mandy, 2
Name, score: fernando, 4
Name, score: jessica, 1
Name, score: prateek, 1
```

Dictionary Modification

- To add entries, use a unique key along with an assignment statement
- Assigning a value to an existing key will overwrite/update the key's associated value

```
scores = {"mandy": 2,
          "fernando": 4,
          "jessica": 1,
          "prateek": 1}

name = input("Enter your name: ")
score = int(input("Enter your score: "))

scores[name] = score

for key in scores.keys():
    print("Name, score:", key + ",", scores[key])
```

OUTPUT:

```
Enter your name: George [Enter]
Enter your score: 0 [Enter]
Name, score: mandy, 2
Name, score: fernando, 4
Name, score: jessica, 1
Name, score: prateek, 1
Name, score: george, 0
```

```
scores = {"mandy": 2,
          "fernando": 4,
          "jessica": 1,
          "prateek": 1}
name = input("Enter your name: ")
score = int(input("Enter your score: "))

scores[name] = score

for key in scores.keys():
    print("Name, score:", key + ",", scores[key])
```

OUTPUT:

```
Enter your name: Jessica [Enter]
Enter your score: 3 [Enter]
Name, score: mandy, 2
Name, score: fernando, 4
Name, score: jessica, 3
Name, score: prateek, 1
```

Contd…

- Call the pop() method on a dictionary to remove an entry, passing in the key for the entry you want to remove as an argument
- The pop() method does not return anything, so to see the changes, you need to print the dictionary.

```
scores = {"mandy": 2,
          "fernando": 4,
          "jessica": 1,
          "prateek": 1}

scores.pop("mandy")

for key in scores.keys():
    print("Name, score:", key + ",", scores[key])
```

OUTPUT:

```
Name, score: fernando, 4
Name, score: jessica, 1
Name, score: prateek, 1
```

Introduction to Pandas DataFrames

- A 2D, table-like data structure (rows and columns)
- Part of Pandas library (must be installed separately)
- Designed for easy data storage, analysis, and manipulation
- Similar to Excel spreadsheets or SQL tables
- Easier than handling multiple lists or dictionaries for tabular data

- Pandas is a Python library used for working with data sets
- Used for analysing, cleaning, exploring and manipulating data

Example

```python
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [20, 21, 19],
    "Grade": ["A", "B", "A"]
}

df = pd.DataFrame(data)
print(df)
```

Output

```
      Name  Age Grade
0    Alice   20     A
1      Bob   21     B
2  Charlie   19     A
```

Accessing DataFrames

Access Columns

```
# Single column
print(df["Name"])
```

```
0      Alice
1        Bob
2    Charlie
```

```
# Multiple columns
print(df[["Name", "Age"]])
```

```
      Name  Age
0    Alice   20
1      Bob   21
2  Charlie   19
```

Access Rows

```
# Row by label
print(df.loc[0])
```

```
Name      Alice
Age          20
Grade         A
Name: 0, dtype: object
```

```
# Row by index
print(df.iloc[1])
```

```
Name      Bob
Age        21
Grade       B
Name: 1, dtype: object
```

Access Specific Element

```
# Row 0, column "Age"
print(df.loc[0, "Age"])
```

```
20
```

```
# Row 1, column index 2
print(df.iloc[1, 2])
```

```
B
```

Traversing a DataFrame

Loop through rows (iterrows)

```
for index, row in df.iterrows():
    print(row["Name"], row["Age"])
```

```
Alice 20
Bob 21
Charlie 19
```

Loop through columns

```
for col in df.columns:
    print(col, ":", df[col].tolist())
```

```
Name : ['Alice', 'Bob', 'Charlie']
Age : [20, 21, 19]
Grade : ['A', 'B', 'A']
```

Common DataFrame Operations

Operation	Description	Example
df.head()	Displays first 5 rows (default)	df.head()
df.tail()	Displays last 5 rows (default)	df.tail()
df.shape	Number of rows and columns	df.shape
df.columns	List of column names	df.columns
df.describe()	Summary statistics	df.describe()
df.info()	Info about data types	df.info()
df.sort_values(col)	Sort rows by column	df.sort_values(“Age”)
df[“col”].mean()	Column average	df[“Age”].mean()

Modifying DataFrame

```python
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [20, 21, 19],
    "Grade": ["A", "B", "A"]
}

df = pd.DataFrame(data)
print(df)
print("\n")

print("Adding a column")
df["City"] = ["Sydney", "Melbourne", "Perth"]
print(df)
print("\n")

print("Removing a column")
df.drop("Grade", axis=1, inplace=True)
print(df)
print("\n")

print("Filtering Rows")
df[df["Age"] > 20]  # Only students older than 20
print(df)
print("\n")

print("Updating a value")
df.loc[0, "Age"] = 21  # Change Alice's age
print(df)
```

Output

```
      Name  Age Grade
0    Alice   20     A
1      Bob   21     B
2  Charlie   19     A


Adding a column
      Name  Age Grade       City
0    Alice   20     A     Sydney
1      Bob   21     B  Melbourne
2  Charlie   19     A      Perth


Removing a column
      Name  Age       City
0    Alice   20     Sydney
1      Bob   21  Melbourne
2  Charlie   19      Perth


Filtering Rows
      Name  Age       City
0    Alice   20     Sydney
1      Bob   21  Melbourne
2  Charlie   19      Perth


Updating a value
      Name  Age       City
0    Alice   21     Sydney
1      Bob   21  Melbourne
2  Charlie   19      Perth
```

Next Week – What's Coming Up?

- Differentiate between built-in functions, imported functions, and programmer-defined functions.
- State the purpose of a function's return value.
- Explain the meaning of "pass by value" and "pass by reference" in the context of functions

- Describe module structures
- Explain how the keyword import copies code from another module.
- Explain how to create modules
- Understand the purpose of the modules in Python

CIHE

CIHE Australia